COMPUTING METHODS IN HIGH ENERGY PHYSICS

S. Lehti

Helsinki Institute of Physics

Spring 2026

Reconstruction

Event reconstruction

Reconstruction is the operation of constructing physics quantities from the raw data collected in the experiment. As a software process, reconstruction is therefore the procedure of data reduction whose main client is data analysis.

The reconstruction process is seen as a collection of independent units, each one providing a set of corresponding objects as output. The reconstruction process can be divided into three steps, corresponding to local reconstruction within an individual detector module, global reconstruction within a whole detector, and combination of these to produce higher level objects.

The reconstruction units providing local reconstruction in a detector module use as input real data from the DAQ system or simulated data representing the real data. These data are in either case called digis. The output from the reconstruction units are RecHits, reconstructed hits which are typically position measurements in tracking type detectors and calorimetric clusters in calorimeter systems. The RecHits are used as the input for global reconstruction.

In the global reconstruction step information from the different modules of a subdetector are

Reconstruction

combined, although information from different subdetectors is not. For example, tracker RecHits are used to produce reconstructed charged particle tracks and muon system RecHits are used to produce candidate muon tracks.

The final reconstruction step combines reconstructed objects from individual subdetectors to produce higher level reconstructed objects suitable for high-level triggering or for physics analysis. For example, tracks in the tracker system and tracks in the muon system are combined to provide final muon candidates, and electron candidates from the calorimeter system are matched to tracks in the tracker system.

Local reconstruction

Local reconstruction leads to RecHits which contain information about the energy deposition and positions.

In the tracker detectors local reconstruction algorithms search for strips and pixels with a signal exceeding a threshold, and use them as seeds for clusters. Clusters are built by adding neighboring strips/pixels.

In the muon drift tubes, local reconstruction provides the position of a muon hit in a drift cell, determined from the drift time measurement and the effective drift

Reconstruction

velocity. Three-dimensional track segments within a superlayer are built from hits in each component layer.

In the muon cathode strip chambers, local reconstruction provides position and time of arrival of a muon hit from the distribution of charge induced on the cathode strips. Two dimensional hits are obtained in each layer, and these can be combined to create three-dimensional track segments within each chamber.

In the muon resistive plate chambers, local reconstruction gives the position of a muon hit from the position of clusters of hit strips.

In the electromagnetic calorimeter (ECAL), local reconstruction identifies the position, time of arrival, and energy of localized electromagnetic energy depositions.

In the hadron calorimeter (HCAL), local reconstruction identifies the position, time of arrival, and energy of localized electromagnetic energy depositions.

Global reconstruction

The global reconstruction algorithms use the object created in the local reconstruction within a single detector module, combining them with objects arising from other modules of the same subdetector to produce further objects which represent the best measurement

Reconstruction

from that subdetector.

- Reconstruction in the tracker system: high/low p_T tracks, displaced vertices etc.
- Reconstruction in the calorimeter system: ECAL+HCAL tower linking to be used as a basis for jet reconstruction
- Reconstruction in the muon system: reconstruction of muons without inner tracker information in an inhomogeneous and nonuniform magnetic field.

Track reconstruction in a dense environment needs an efficient search for hits during the pattern recognition stage and a fast propagation of trajectory candidates. The track reconstruction is decomposed into five logical parts:

- hit reconstruction, which in turn consists of clustering of strips or pixels and estimating a position and its uncertainty.
- seed generation providing the initial trajectory candidates for the full track reconstruction.
- pattern recognition or trajectory building
- ambiguity resolution
- final track fit

Ambiguities in track finding arise because a given track may be reconstructed starting from different

Reconstruction

seeds, or because a given seed may result in more than one trajectory candidate.

Vertex reconstruction (interaction points) usually involves two steps, vertex finding and vertex fitting. Vertex finding involves grouping of tracks into vertex candidates. The vertex finding algorithms can be very different depending on the physics case (primary or secondary vertex, reconstruction of exclusive decays, etc.). Vertex fitting involves determining the best estimate of the vertex parameters (position, covariance matrix, track parameters constrained by the vertex position and their covariances) for a given set of tracks, as well as indicators of the fit quality.

Combined reconstruction

The final stage of reconstruction combines input objects created in the global reconstruction within each subdetector, creating objects based on the complete detector.

Photon and electron identification. The global selection of electrons and photons proceed in three steps. The first step uses the calorimeter information only. The second step requires hits in the pixel detectors, consistent with an electron candidate. The success of the matching ECAL supercluster to hits in the pixel detector flags the candidate as an electron, otherwise

Reconstruction

the candidate is flagged as a photon. In the final step, the selection of electrons uses full track reconstruction, seeded from the pixel hits obtained by the matching step. The selection of photons can instead use isolation cuts and rejection of π^0's based on lateral shower shape and the reconstruction of converted photons.

Muon identification. Global muon identification starts from the standalone muon, adding associated silicon tracker hits and performing final fit to the track. Isolation criteria can be applied to the muon candidates to provide additional rejection against nonprompt muons from b, c and K decays.

Jet reconstruction. Jet reconstruction aims to reconstruct and identify jets arising from the hadronization of a scattered parton, in order to reconstruct its direction and energy. Many reconstruction algorithms exist in the literature, varying in speed, efficiency and resolution. Most algorithms use a clustering technique, in which calorimetric towers close in (η, φ) to a high $\mathrm{E_T}$ tower are summed together, subject to some constraints. For example in the cone algorithm, a seed tower is selected and then all objects sufficiently close in (η, φ) are used to form a protojet. The process of association is iterated until the parameters of the

Reconstruction

protojet have stabilized. The procedure is repeated with the remaining unassociated towers, until no seeding tower with sufficiently high E_T remains.

MET reconstruction (missing E_T). Many channels of discovery at the LHC present as a clear signature for new physics a large missing transverse energy. Missing transverse energy is typically reconstructed by summing the energy depositions times $\sin\theta$ over the whole calorimeter system, and taking the transverse momentum carried by the muons into account.

b and τ tagging

Many physics channels produce b jets and τ jets in the final state. These need to be distinguished from more copious backgrounds containing only light flavoured jets. The top quark, for example, decays almost exclusively into a W boson and a b quark.

Inclusive tagging of b jets, as opposed to lighter flavour jets, mainly relies upon relatively distinct properties of b-hadrons such as proper lifetime (lifetime at rest), large mass, decays to final states with high charged track multiplicity, relatively large semileptonic branching ratios and a hard fragmentation function.

The b-tagging algoritms rely on the reconstruction of lower level physics

Reconstruction

objects. At the LHC b tagging typically will be applied to jets. Most of the b hadron properties used for b tagging are exploited using charged particle tracks. To increase the signal-to-noise ratio, only tracks fulfilling certain quality criteria are selected.

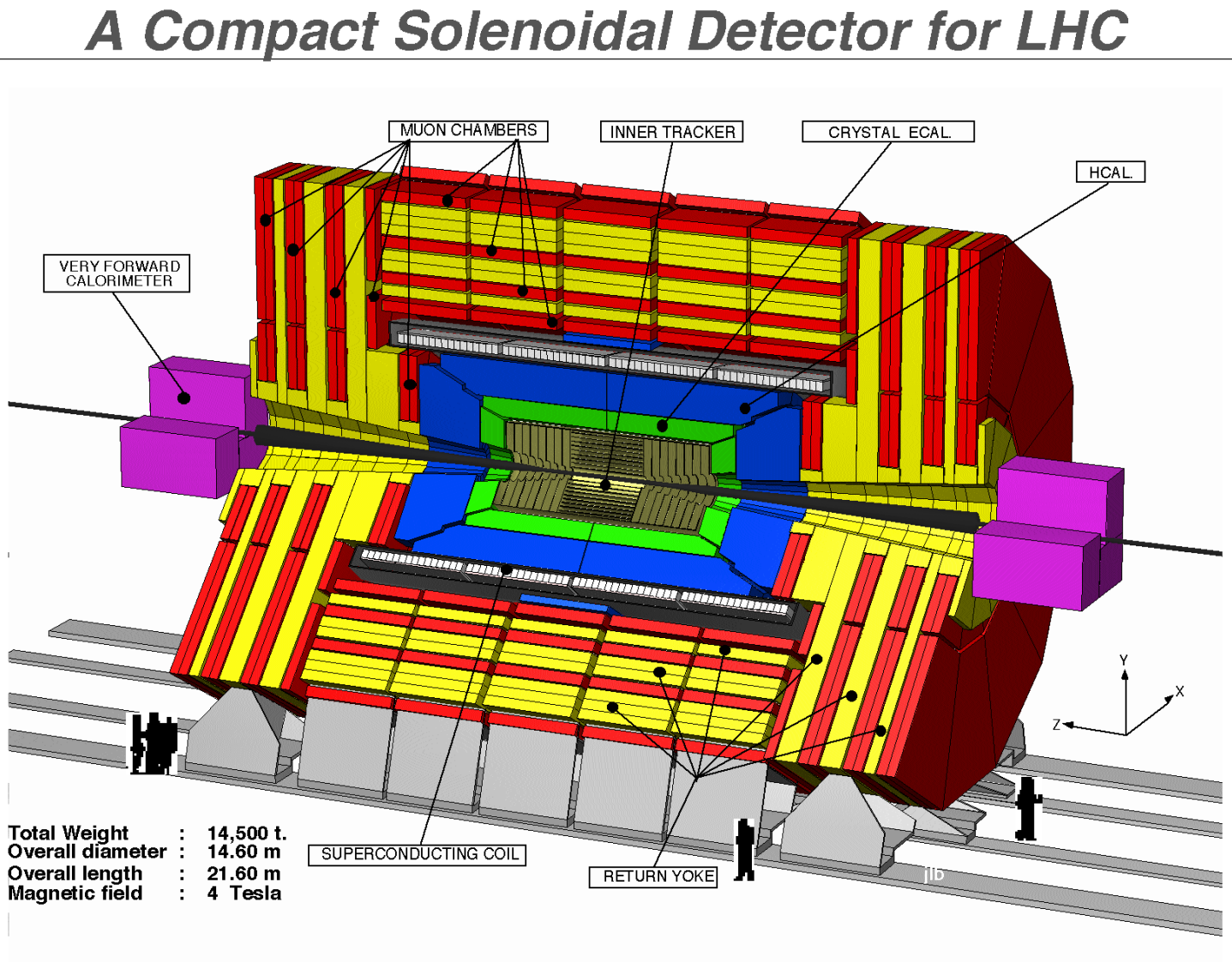
For b tagging the track measurement precision close to the interaction point is most relevant. The simplest b tagging algorithm counts high quality tracks in a cone around the jet axis with high enough *impact parameter* significance. More sophisticated algorithms are described in the literature.

Tau identification is based on τ jet properties such as lifetime, mass, small charged track multiplicity, collimation and isolation of τ decay products. Tau jet identification is important, since τ lepton decays hadronically about 65% of the time producing a τ jet. In general, the primary requirement for τ jet identification is the isolation of a collimated group of charged particles in the tracker.

Trigger

Trigger is the start of the physics event selection process. For the nominal LHC design luminosity of $10^{34}\mathrm{cm}^{-2}\mathrm{s}^{-1}$, an average of 17 events occurs at the beam crossing frequency of 25 ns. This input rate of 10^9 interactions every second must be reduced by a factor of at

Reconstruction



Reconstruction

least 10^7 to 100 Hz, the maximum rate that can be absorbed by the on-line computer farm.

The CMS Level-1 (L1) trigger system is based on custom electronics. The High Level Trigger (HLT) system relies on software.

The CMS L1 trigger is based on the presence of local objects such as muons, electrons, photons and jets, and it employs global sums of E_T and MET. The requirements on the L1 trigger are chosen to provide a high efficiency for the hard scattering physics to be studied at the LHC. The L1 trigger operates without the benefit of full detector granularity and full-resolution data.

The HLT is used to provide the selection of 1:1000 from the maximum average Level-1 output rate of 10^5 Hz to the final storage rate of 100 Hz. The HLT has a total processing time of up to $\sim$1 s, during which time interval the data are stored in memory.

Table: Example trigger thresholds

Trigger	Threshold (GeV)
Inclusive electron	29
Di-electrons	17
Inclusive photons	80
Di-photons	40,25
Inclusive muon	19
Di-muons	7
Inclusive τ-jets	86
Di-τ-jets	59
1-jet + MET	180 + 123
1-jet OR 3-jets OR 4-jets	657,247,113
Electron+τ-jet	19 + 45
Inclusive b-jets	237

Reconstruction

To minimize the CPU requirement by the HLT, a key feature of the algorithms is to reconstruct the information in the CMS detector only partially. An example set of thresholds for some trigger objects are shown in Table 1.

The trigger algorithms can be simulated and their efficiences studied by simulation. To have a realistic physics analysis, the trigger simulation needs to be included in the event selection.

Reconstruction software in CMS

Although the used reconstruction algorithms and methods are general and used widely in HEP, the reconstruction software is highly experiment dependent. The reconstruction sotfware is written by the physicists for their own use. The analysis software is even more specific: as the reconstruction is done the same way within an experiment, the analysis using the reconstructed objects varies from physics channel to physics channel.

Here we do not go into the CMS reconstruction software, but describe only the basic principles how to reconstruct the physics objects for your own analysis.

The first place to start is the documentation. Although it may be in many cases almost non-existent, it is an obvious place to start.

Reconstruction

Use tutorials and examples as working examples from which you can take the relevant part or enlarge to fulfill your needs.

Go to the source code and look for validation code and examples available. This is often a very fruitful approach.

. . . And most of all, ask help from the experts responsible for writing/maintaining the software in your experiment, or from other users who may have already solved your problem and who may have code available which you can easily adopt to your study.

How to browse the source code in CMS? The first way is using the GitHub browser, or the reference manual. The source can also be accessed directly via scram,

```
scram list
```

Here you see the paths to the source code, go to the source code directory and start digging with ls, grep, find | xargs grep etc. If you find something interesting, check out (git cms-addpkg) the code, and test it. Sometimes it doesn't work, and in such a case you did not find a *working* example, and you need to keep searching . . .

The config for running the DIGI and RECO steps in CMSSW are created with cmsDriver.py:

Reconstruction

Digi step:

```
cmsDriver.py digi -s
DIGI,L1,DIGI2RAW,HLT:GRun
--conditions auto:run2_mc_GRun
--eventcontent RAWSIM --datatier
GEN-SIM-RAW -n -1 --no_exec
--filein file:fin.root --fileout
fout.root
```

RECO step:

```
cmsDriver.py reco -s
RAW2DIGI,RECO --filein
file:fin.root --fileout
fout.root --conditions
auto:run2_mc_GRun --no_exec -n -1
```

Fast simulation

Introduction

Many simulation tasks, such as extracting rare signal events from the expected background, require a generation of millions of background events, of which only a handful survives after a proper selection. If detailed detector simulation is too time consuming to be applied, while still needing some reasonable estimates for the detector response, the solution can be found by parametrizing the detector response. The parametrized detector response simulation is called fast simulation.

Parametrizing and smearing provide an approximate but very fast method to simulate the detector response. It is a useful tool for quick first inspection of properties of a given physics channel with large statistics.

Single particle performance

The key parameters for tracker resolution for single particles are

- transverse momentum p_T
- azimuthal angle ϕ_0
- polar angle θ
- transverse impact parameter d_0
- longitudinal impact parameter z_{ip}

The transverse impact parameter is defined as the closest point of approach to the impact reference point (primary vertex).

Fast simulation

The resolution is studied with full simulation as a function of track p_T and η. The resolutions are fitted as a function of η for each value of p_T using polynomial functions up to power five, which is needed to describe the most irregularly shaped η-dependencies. These functions are used to give the resolution for fixed values of η in order to have points of resolution as a function of p_T.

An approximate parametrization of the p_T dependence of the impact parameter and angular resolution is expressed as

$$\sigma_{ip,ang}(p_T) = (a_1 \oplus a_2/p_T) + a_3$$

Likewise, a first approximation of the p_T resolution dependence on p_T is of the form

$$\sigma_{p_T}(p_T) = b_1 \oplus b_2 p_T$$

where the first term corresponds to the resolution due to the finite detector precision and the second term the uncertainty due to multiple scattering.

In order to improve the fits, one may use a "covariance" or cross term, e.g. for impact parameter

$$\sigma_{d_0}(p_T) = \sqrt{a_1^2 + a_4/p_T + a_2^2/p_T^2} + a_3$$

In the functions the coefficients are calculated for discrete values of η_j. The resolution for arbitrary η is found by linear interpolation (extrapolation) from two points of

Fast simulation

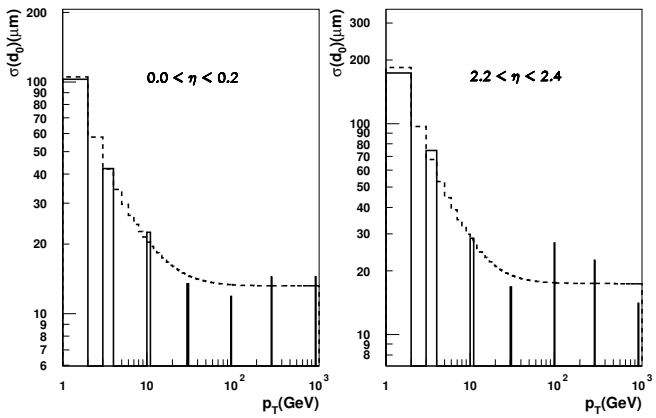


Figure B.1: An example of the transverse impact parameter fitted by the least squares method as a function of track p_T for two different η regions (central barrel and the edge of the end-cap).

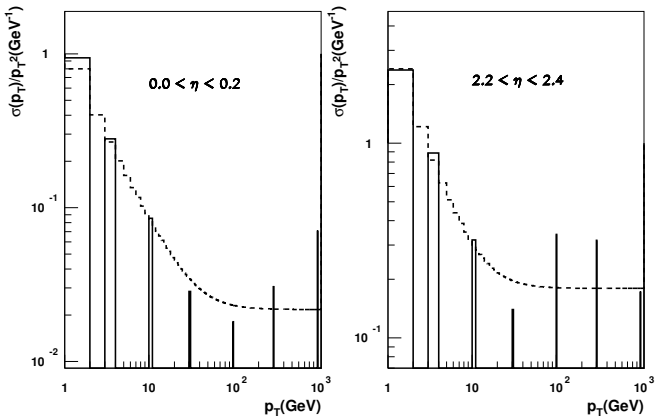


Figure B.2: The p_T resolution fitted as a function track p_T for two different η-regions.

$\sigma(\eta_j)$ with η_j's closest to η, and used in smearing the ip.

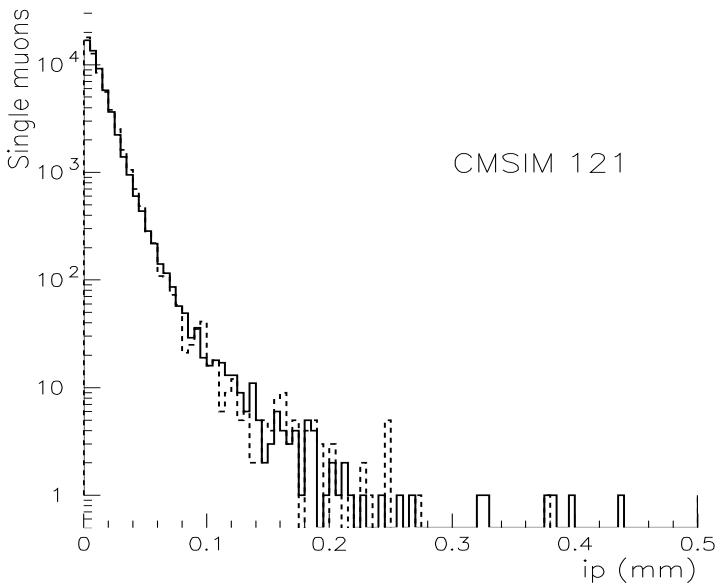


Figure: CMSIM (solid), and parameterized and smeared (dashed) ip.

Fast simulation

Particle showers

Calorimeters are described in terms of characteristic parameters such as energy resolution, position resolution, and shower separation, dynamical range and spatial coverage. A calorimeter can be homogeneous (like crystal calorimeter) or sampling calorimeter (sandwich structure with active and inactive layers).

The energy resolution of calorimeters is usually expressed in terms of $\Delta E/E$, ΔE being the overall statistical error including all error sources. At high energies, statistical contributions dominate. The relative resolution improves with increasing E. The main tendency for $\Delta E/E$ is to follow the form $k/\sqrt{E}$ where k depends on the calorimeter and readout characteristics. A general formula to get an approximate resolution is

$$\Delta E/E = \sqrt{A^2 + B^2} \qquad (1)$$

where A is related to photon statistics in the photomultipliers, and B deals with the sampling fluctuations.

Containment describes the fraction of energy measured in the calorimeter. Energy not contained in the calorimeter may result in severe biases or simply be detrimental to the energy resolution.

Fast simulation

A detailed simulation of electromagnetic showers is computationally intensive. A parameterization of the spatial energy distribution of an electromagnetic shower, based on probability density functions allows speeding the simulation without compromising the simulation accuracy. The parameterized simulation is tuned to give the same results as the full detector response simulation.

CMS Fast Simulation

CMS Fast Simulation is a CMSSW-integrated tool to simulate and reconstruct events with the CMS detector, in view of doing physics analysis without being penalized by either CPU time or disk space considerations, while still benefiting from an accurate simulation of the detector effects. The input of Fast Simulation is a list of particles from an event generator, propagated in the magnetic field to different layers of the various subdetectors, with which the particles may interact. The quasi-stable particles are allowed to decay.

The interactions simulated with Fast Simulation are

- electron bremsstrahlung
- photon conversion
- charged particle energy loss by ionization

Fast simulation

- multiple scattering
- electron, photon and hadron showering

The first 4 are applied to particles traversing the thin layers of the tracker, while the latter is parameterized in the electromagnetic and hadron calorimeters.

The muon simulation is mostly based on parameterization of resolutions and efficiencies.

As output, Fast Simulation delivers a series of "high-level objects", such as reconstructed hits for charged particles in the tracker layers and energy deposits in calorimeter cells, which can be used as inputs for the reconstruction algorithms.

The computer time needed to simulate an event in Fast Simulation is about 3 orders of magnitude smaller than that needed in the full simulation.

Tracker response

Fast Simulation uses a simplified tracker geometry, adequate for the required level of accuracy. The thickness of the active layers is tuned to reproduce the fully simulated number of bremsstrahlung photons above a certain energy threshold. After tuning, the total number of radiation lengths traversed in the tracker is in agreement with the full geometry.

While propagating in the magnetic

Fast simulation

field through tracker layers, charged particles experience multiple scattering and energy loss by ionization. The intersections between the modified trajectories and each tracker layer define the position of "simulated hits". Each simulated hit is turned with a certain efficiency to a "reconstructed hit". The position is obtained from a gaussian smearing of the simulated hit position. The gaussian resolution used in the smearing process is obtained from the full simulation for each silicon strip and pixel layer. The detailed procedure was developed in view of reproducing the b-tagging performance with requested level of accuracy.

Calorimeter response to e/γ

The showers of electrons and photons which impinge on the ECAL are simulated in 2 steps. In the first step, the shower is developed following a parameterization as if ECAL were a homogeneous medium. This approximation is realistic since the CMS ECAL is made of contiguous crystals. The deposited energy is integrated over longitudinal slides including uncertainties due to the limited photo-statistics and the longitudinal non-uniformity in the crystals.

In the second step, the energy spots in each slide are distributed in space and placed into the actual crystal

Fast simulation

geometry. The time used in this step is kept to a reasonable value by limiting the 2D spot-crystal assignment to a small 7x7 crystal grid in a plane perpendicular to the shower longitudinal component. The energy collection is then simulated taking into account energy leakage, losses due to gaps in the geometry, and shower enlargement due to the magnetic field.

When all electrons and photons are processed, the electronic noise is simulated and zero suppression applied, and a list of reconstructed hits is built. The fast and full simulations agree at the level of per mil in the barrel region, and at the level of per cent in the endcaps, for energies ranging from 1 GeV to 1 TeV.

Calorimeter response to hadrons

Charged and neutral hadrons are propagated to the ECAL, HCAL and forward calorimeter entrance after their interactions with the tracker layers. Their energy response is derived from the full simulation. Single charged pions are fully simulated for p_T values of 2, 5, 10, 30, 50, 100 and 300 GeV, uniformly distributed in pseudorapidity between -5 and 5. The reconstructed energy is collected in 5x5 HCAL tower matrices and in the corresponding 25x25 ECAL crystals. The energy distributions are then sliced into η bins. These

Fast simulation

distributions are fitted to a gaussian, the mean value and the sigma of which are tabulated as a function of the energy and pseudorapidity, used in turn to smear the hadron energy response in the fast simulation. Linear interpolation and extrapolation is used.

Other hadrons are simulated as if they were pions with the same transverse energy. The agreement between fast and full simulations is satisfactory.

Response to muons

The response of the muon chambers is simply parameterized to reproduce the efficiencies and resolutions of the full simulation. At the moment, the detailed particle propagation stops at the entrance of the calorimeters. The muon interactions with calorimeters is simulated much as the pion interactions. The fast muon simulation results agree within 1.5% with the full simulation results.

Usage

Fast Simulation source code is available as part of the CMSSW code. The documentation can be found under the link *https://twiki.cern.ch/twiki/bin/view/CMSPublic/SWGuideFastSimulation* Unfortunately part of the documentation is CMS only with passwd protection.

Fast simulation

Start Fast Simulation simulation by creating the working area. First the environment needs to be set to get access to scram. We can take the latest available version on kale by

scram p CMSSW CMSSW_16_0_4

&& cd CMSSW_16_0_4/src

The GEN-fragments for official CMS production are located in the git directory

Configuration/Generator

In addition one needs to check-out any code which is called by the GEN-fragment, like the undelying event parameters. The configuration file for CMSSW is generated from the GEN-fragment with cmsDriver.py:

```
cmsDriver.py GENfragment_cfi.py
-s GEN,SIM,RECOBEFMIX,DIGI,L1,L1Reco,
RECO,HLT:@frozen25ns
--fast --conditions auto:run3_mc_GRun
--eventcontent AODSIM
--beamspot DBrealistic
--era Run3_2026
--datatier AODSIM
-n 10 --no_exec
```

```
cmsRun
GENfragment_cfi_py_GEN_SIM..._HLT.root
```

Even if the GEN-fragments available in the repository do not contain your favourite channel, they can be used as working examples.

In Fast Simulation the event generator input can be generated and fed to the analysis on the fly or

Fast simulation

one can store the events in ROOT files for later analysis with a separate analysis program.

There are example config files in the directory

`FastSimulation/Configuration/test`

which can be customized for user's needs by making some modifications. One can change for example the average number of pile-up events, simulate (or not) hits in the tracking system and calorimetry, etc: ...

```
process.maxEvents=
    cms.untracked.PSet(input=cms.untracked.int32(5))
...
# If you want to turn on/off pile-up
process.famosPileUp.PileUpSimulator.averageNumber=5.0
# If you want to choose simulated detectors
process.famosSimHits.SimulateCalorimetry =
True
process.famosSimHits.SimulateTracking = True
# process.famosSimHits.SimulateMuons = False
...
```

Remember that fast simulation is not a full simulation, it gives more approximate results, and it cannot always be used. Sometimes it is, however, the only practical option, and often a very good starting point for a "first look" in a new physics channel.